CS 422 Assignment 2: iPerf, TCP Statistics, and Congestion Control

Thanh Bui Minh (tbuiminh) Aaruni Gupta (gupt1066) Rishiraj Behki (rbehki)
Pranith Ravella (pravella) Colin Hermack (chermack)

Fall 2026

Repository: https://github.com/ColinHermack/CS422-Project2

AI tools acknowledgement

OpenAI Codex and Anthropic Claude were used to assist with code generation, testing and debugging, measurement analysis, and drafting and polishing this report. All code, results, interpretations, and design choices were reviewed and verified by the group. We understand the submitted work and are prepared to explain the implementation, measurements, and congestion-control design during evaluation.

Code reference index

The links below go directly to the relevant implementation in the pinned commit used for this report. More specific function-level links also appear throughout the corresponding sections.

Report requirement	Relevant code
iperf3 protocol and complete client transaction	`run_experiment` and its protocol steps
Random server selection and live usability probes	`select_destinations`
`TCP_INFO` decoding and goodput calculation	`decode_tcp_info` / `compute_goodput_bps`, `take_sample`
Command-line options and single-entry-point pipeline	`parse_args`, `main`
Summary tables and PDF plot generation	`generate_report`
Ubuntu 24.04 container and automated Linux/WSL2 launcher	`Dockerfile`, `run_wsl_docker.sh`

1. iPerf Throughput Application

1(a) iperf3 client from scratch

`iperf3_client.py` is a TCP client for the standard iperf3 server written with Python's `socket`, `struct`, and `json` modules only. It never invokes the `iperf3` binary and never uses a custom receiver: every test in `results/20261021-152852` ran against an unmodified public server from iperf3serverlist.net. We reconstructed the protocol from iperf3's source (`src/iperf_api.h`, `iperf_api.c`, `iperf_client_api.c`); the sequence is summarized in the module docstring.

Wire format. The server drives the test by sending one signed byte, a state code, on the control connection; the client answers some states with its own code. The codes are named constants copied from `iperf_api.h` (L42-L56), e.g. `PARAM_EXCHANGE = 9`, `CREATE_STREAMS = 10`, `ACCESS_DENIED = -1`. JSON messages are framed as a 4-byte big-endian length followed by the UTF-8 JSON text, mirroring iperf3's `JSON_write` (`send_json` / `recv_json`). `recv_exact` loops until it has the requested number of bytes, since `recv()` may return less (L287-L314).

Protocol steps. One function per step, called in order by `run_experiment`:

Spec step	What the client does	Code
(i) Control connection	TCP connect to the server, send a 37-byte cookie (36 random characters from iperf3's alphabet plus NUL). The server uses the cookie to match the later data connection to this test.	`connect_control`
(ii) Parameter exchange	Wait for `PARAM_EXCHANGE`, send the test parameters as JSON (`tcp`, `time`, `parallel = 1`, `len = 131072`, ...: the keys iperf3's own `send_parameters` sends), then wait for `CREATE_STREAMS`, which means the server accepted them.	`exchange_params`
(iii) Data connection	Open a second TCP connection to the same IP and port, set `TCP_CONGESTION` on it **before** `connect()`, send the same cookie, and read the algorithm back with `getsockopt`. Then wait for `TEST_START` and `TEST_RUNNING`.	`open_data_stream`, `wait_for_test_running`
(iv) Timed send	Send 128 KiB blocks for `--duration` seconds on a non-blocking socket, using `select()` so sending never delays a `TCP_INFO` sample (Section 2(a)). The control socket is watched at the same time so a server abort is noticed immediately.	`send_and_sample`
(v) Termination	Send `TEST_END`; on `EXCHANGE_RESULTS` send our results JSON (every key the server's `get_results` requires, or it rejects the message) and read the server's; on `DISPLAY_RESULTS` send `IPERF_DONE` and close both sockets.	`exchange_results`, `finish_test`

A test counts as successful only after the full termination handshake. As an end-to-end check, the server's own count of bytes received (from its results JSON) agrees with our final `tcpi_bytes_acked` to within 0.2% (median) and 2.7% (worst case) across all 89 completed runs. In 86 runs the server's count is slightly larger, because data still in flight at our last sample reaches the server but is not yet acknowledged to us; in the other 3 it is up to 0.7% smaller, since the two ends stop counting at slightly different moments.

Robustness. Every failure becomes an `IperfError` with a readable reason, so the caller can log it and move on; a busy server raises the subclass `ServerBusyError`, because the fix is different (try another port). On any failure, `run_experiment` sends `CLIENT_TERMINATE` (best effort) so the server frees itself sooner, and closes both sockets (L811-L840).

Failure	How it is detected	Handling	Seen in our campaign
Unresponsive or overloaded server	Every socket has a 10 s timeout (`DEFAULT_TIMEOUT`); a connect or control read that exceeds it raises `TimeoutError`.	Converted to `IperfError("timed out ...")`.	2 hosts during selection; 6 failed tries during the campaign, all succeeded on retry.
Busy server (one test at a time)	Server answers `ACCESS_DENIED` (-1) instead of `PARAM_EXCHANGE`.	`ServerBusyError`; selection tries another port on the same host.	2 hosts during selection; the one failed campaign run (below).
Rejected parameter negotiation	Server sends `SERVER_ERROR` (-2) instead of `CREATE_STREAMS`, followed by its iperf3 error code and errno, which we read and log.	`IperfError` with both codes.	Not seen.
Connection timeout or refusal	`connect()` times out, or raises `ConnectionRefusedError` / `ENETUNREACH` (all `OSError`).	`IperfError("socket error: ...")`.	1 host (network unreachable).
Premature termination	`recv()` returns 0 bytes (server closed), a reset raises `OSError`, or the control socket becomes readable mid-test with `SERVER_TERMINATE` (`handle_control_message_during_test`).	`IperfError`.	2 hosts during selection (closed, reset); 1 campaign try aborted mid-send, succeeded on retry.
Malformed reply	Length prefix over 1 MiB, invalid UTF-8/JSON, JSON that is not an object, or an unknown state code.	`IperfError`; never crashes on server input.	Not seen.

1(b) Destination selection

`destinations.py` reads `listed_iperf3_servers.csv` (185 servers from iperf3serverlist.net). The `PORT` column is a single port or an inclusive range such as `5201-5210`; `parse_ports` expands ranges, since such hosts run one iperf3 server per port.

`select_destinations` shuffles the list (optionally with `--seed` for a reproducible order) and probes candidates with a real 10 s iperf3 test, keeping the first `n` that complete it. A probe is a full test, not just a TCP connect, because many listed servers accept connections but then refuse or abort the test; 10 s also outlasts iperf3's 5 s `--server-bitrate-limit` averaging window, so a rate-limited server fails the probe instead of every real test. For each host, a busy answer moves on to another random port (up to 3 per host); any other failure skips the host. The total number of probes is bounded at `3 * n` (30 for `n = 10`), after which any shortfall is logged and saved in `metadata.json` (`destination_shortfall`).

For the campaign, 17 probes found all 10 destinations (no shortfall). The 7 failed probes, recorded in `metadata.json` (`selection_failures`):

Host:port	Reason
69.48.236.198:30007	server closed the connection
man.speedtest.clouvider.net:5209	busy (`ACCESS_DENIED`); tried another port
man.speedtest.clouvider.net:5205	connection reset by peer; host skipped
169.150.252.2:5201	busy (`ACCESS_DENIED`)
speedsrv.multinet24.pl:5302	timed out after 10 s
102.130.68.171:9210	timed out after 10 s
speed-cb.dimensione.com:5204	network unreachable

Selected destinations (address and port as tested; IP is what the hostname resolved to):

Host	Port	IP	Country	Provider
156.146.40.65	5201	156.146.40.65	SK	DATAPACKET
speedtest.wdc2.us.leaseweb.net	5205	23.105.9.80	US	LeaseWeb
123.53.44.221	5215	123.53.44.221	CN	China Telecom
iperf.pendc.com	5208	62.106.94.245	TR	PENDC
speedtest6.cwpanama.net	5508	201.225.207.3	PA	CW Panama
la.speedtest.clouvider.net	5205	77.247.126.223	US	Clouvider
ping-ams1.online.net	5209	163.172.208.7	NL	Scaleway
189.25.101.151	30007	189.25.101.151	AE	FortiSASE
speedtestfl.telecom.mu	5209	197.227.12.18	MU	Mauritius Telecom
speed1.fiberby.dk	9213	193.106.164.10	DK	Fiberby

Failures after selection. A server can pass its probe and be busy later. Every algorithm must be measured over the same destinations (Part 3), so a failed test is retried on the *same* server rather than replaced: up to 3 tries, 10 s apart (`run_with_retries`), with a 3 s pause before every test because many public servers restart their iperf3 process after each test and refuse connections briefly (L44-L59). Five runs needed a retry and then succeeded. One run, `la.speedtest.clouvider.net:5205` with bbr in repetition 2, got `ACCESS_DENIED` on all 3 tries and is recorded as failed, so 89 of 90 runs have data. A destination where *every* run failed would count as a shortfall after selection (`destinations_without_data`, L580-L606); there were none.

1(c) Goodput

Every `--interval` seconds (1 s in our campaign), the client reads `TCP_INFO` from the **data** socket with `getsockopt` and computes goodput from the change in the cumulative acknowledged-byte counter `tcpi_bytes_acked` over the measured time between samples (`compute_goodput_bps`, called from `take_sample`):

$$\text{goodput}_i = \frac{\texttt{bytes_acked}_i - \texttt{bytes_acked}_{i-1}}{t_i - t_{i-1}} \cdot 8 \quad \text{[bits/s]}$$

Acknowledged bytes count only data the server actually received, not data still in our send buffer, and count a retransmitted segment once. The measured $t_i - t_{i-1}$ (from `time.monotonic()`, taken right after `getsockopt`) is used instead of the nominal interval, so a late sample does not inflate goodput. Section 2(a) describes the sampling loop and the `struct tcp_info` decoding in detail.

Generated by `plot_goodput_over_time`. All panels share the same axes so destinations can be compared directly.

The summary table of minimum, median, mean, and 95th-percentile goodput for each destination and algorithm is in Section 3(a), and also saved as `results/20261021-152852/goodput_summary.csv`. It is computed by `summarize_goodput`, which pools the 1 s samples of all repetitions of a destination and algorithm (180 samples for 3 runs of 60 s). The 95th percentile uses `statistics.quantiles(values, n=100, method="inclusive")[94]`, i.e. linear interpolation between the closest ranks, treating the samples as the whole population so the result always lies between min and max (`percentile_95`).

2. TCP Stats Tracing During Transfer

All measurements, plots, and observations in this section use **CUBIC**, the default congestion control on the measurement host (`net.ipv4.tcp_congestion_control = cubic`). The client still selects `cubic` explicitly on the data socket and records the algorithm the kernel reports back (`algorithm_used` in `samples.csv`);

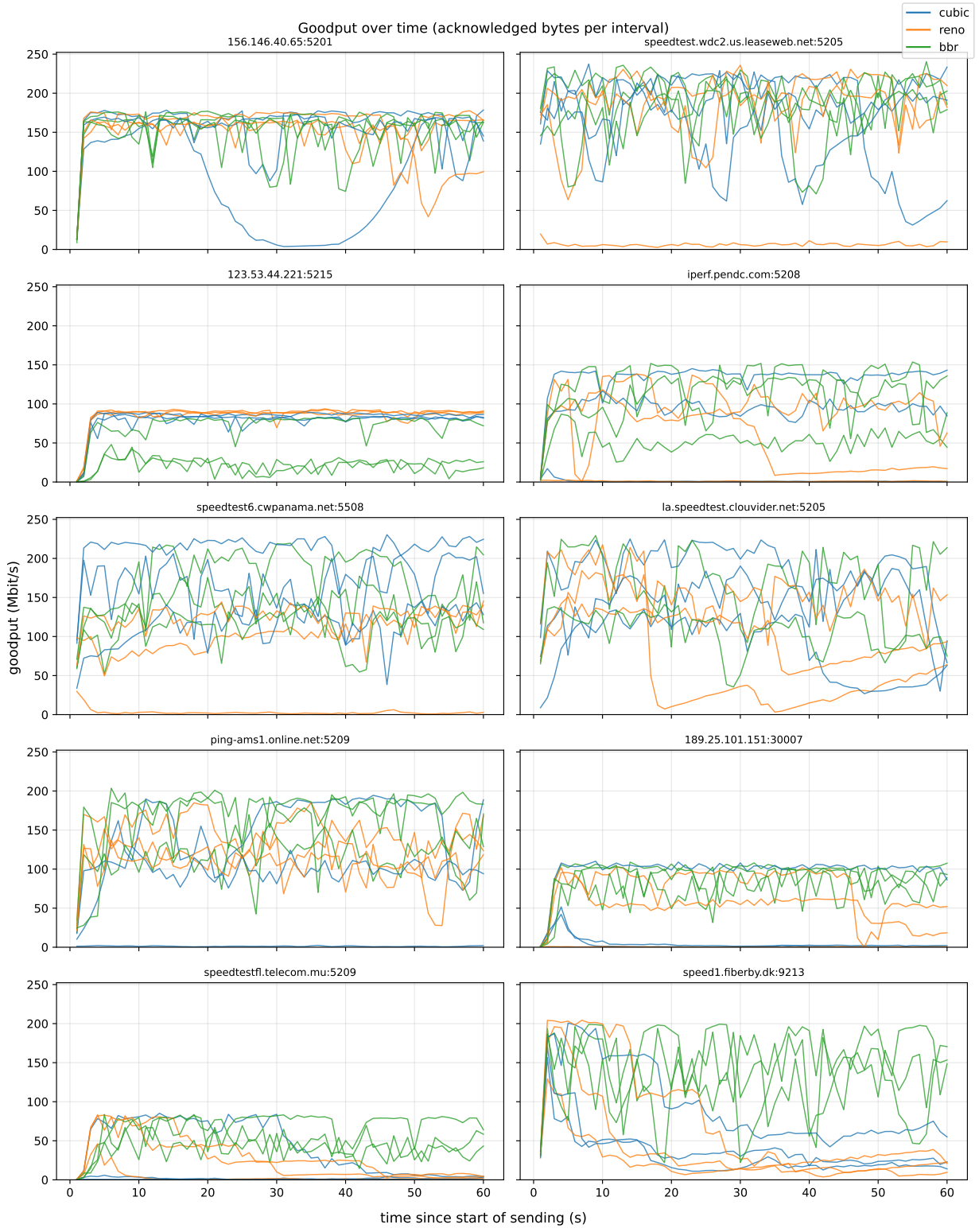


Figure 1: Goodput over time, one panel per destination (shared axes). Each line is one run; color is the algorithm, so the three repetitions of an algorithm share a color. Generated by `report.plot_goodput_over_time`.

every run used `cubic`. The data is the cubic subset of the full campaign in `results/20261021-152852` (10 destinations, 60 s duration per test, 1.0 s sampling interval, 3 repetitions; test order: for each repetition, for each destination, every algorithm back to back).

2(a) Extracting TCP socket statistics

Source socket. Statistics come from the data socket, the TCP connection that carries the test payload, never from the iperf3 control socket. `take_sample` calls `read_tcp_info`, which runs `getsockopt(IPPROTO_TCP, TCP_INFO)` on the data socket and decodes the returned `struct tcp_info` with `decode_tcp_info`.

Host kernel. From `results/20261021-152852/metadata.json` (`host_kernel` is `uname -r`):

```
host_kernel   = 6.18.54.1-microsoft-standard-WSL2
host_platform = Linux-6.18.54.1-microsoft-standard-WSL2-x86_64-with-glibc2.35
```

The kernel's default congestion control is `cubic`, with `reno`, `cubic`, and `bbr` all available (`kernel_tcp_settings`). The kernel-reported `struct tcp_info` length is 280 bytes (`tcp_info_kernel_size_bytes`); we decode the first 232 bytes as described below.

Decoding format. The layout is Linux's `struct tcp_info` from `include/uapi/linux/tcp.h`. We decode the first 232 bytes, from `tcpi_state` through `tcpi_snd_wnd` (the last field added in Linux 5.4), with the Python `struct` format string

```
=BBBBBBBBIIIIIIIIIIIIIIIIIIIIIIIIQQQQIIIIIIQQQQIIQQIIII
```

`=` means native byte order with no automatic padding; this matches the kernel struct because every field is already naturally aligned (each `__u64` starts at a multiple of 8). Kernels newer than 5.4 append more fields after `tcpi_snd_wnd`, so we request a 256-byte buffer and decode only the 232-byte prefix; a kernel that returns fewer than 232 bytes is rejected. The field list, format codes, and units live in one table in the code, `TCP_INFO_FIELDS`, so the decoder and the units saved in `metadata.json` cannot drift apart.

Offset (bytes)	Field	C type	struct code	Unit
0	`tcpi_state`	`__u8`	`B`	code
1	`tcpi_ca_state`	`__u8`	`B`	code
2	`tcpi_retransmits`	`__u8`	`B`	count
3	`tcpi_probes`	`__u8`	`B`	count
4	`tcpi_backoff`	`__u8`	`B`	count
5	`tcpi_options`	`__u8`	`B`	bit flags
6	`tcpi_wscale`*	`__u8`	`B`	bit fields
7	`tcpi_app_limited_flags`*	`__u8`	`B`	bit flags
8	`tcpi_rto`	`__u32`	`I`	microseconds
12	`tcpi_ato`	`__u32`	`I`	microseconds
16	`tcpi_snd_mss`	`__u32`	`I`	bytes
20	`tcpi_rcv_mss`	`__u32`	`I`	bytes
24	`tcpi_unacked`	`__u32`	`I`	segments
28	`tcpi_sacked`	`__u32`	`I`	segments
32	`tcpi_lost`	`__u32`	`I`	segments
36	`tcpi_retrans`	`__u32`	`I`	segments
40	`tcpi_fackets`	`__u32`	`I`	segments
44	`tcpi_last_data_sent`	`__u32`	`I`	milliseconds
48	`tcpi_last_ack_sent`	`__u32`	`I`	milliseconds
52	`tcpi_last_data_recv`	`__u32`	`I`	milliseconds
56	`tcpi_last_ack_recv`	`__u32`	`I`	milliseconds
60	`tcpi_pmtu`	`__u32`	`I`	bytes
64	`tcpi_rcv_ssthresh`	`__u32`	`I`	bytes
68	`tcpi_rtt`	`__u32`	`I`	microseconds

Offset (bytes)	Field	C type	struct code	Unit
72	tcpi_rttvar	__u32	I	microseconds
76	tcpi_snd_ssthresh	__u32	I	segments
80	tcpi_snd_cwnd	__u32	I	segments
84	tcpi_advmss	__u32	I	bytes
88	tcpi_reordering	__u32	I	segments
92	tcpi_rcv_rtt	__u32	I	microseconds
96	tcpi_rcv_space	__u32	I	bytes
100	tcpi_total_retrans	__u32	I	segments
104	tcpi_pacing_rate	__u64	Q	bytes/second
112	tcpi_max_pacing_rate	__u64	Q	bytes/second
120	tcpi_bytes_acked	__u64	Q	bytes
128	tcpi_bytes_received	__u64	Q	bytes
136	tcpi_segs_out	__u32	I	segments
140	tcpi_segs_in	__u32	I	segments
144	tcpi_notsent_bytes	__u32	I	bytes
148	tcpi_min_rtt	__u32	I	microseconds
152	tcpi_data_segs_in	__u32	I	segments
156	tcpi_data_segs_out	__u32	I	segments
160	tcpi_delivery_rate	__u64	Q	bytes/second
168	tcpi_busy_time	__u64	Q	microseconds
176	tcpi_rwnd_limited	__u64	Q	microseconds
184	tcpi_sndbuf_limited	__u64	Q	microseconds
192	tcpi_delivered	__u32	I	segments
196	tcpi_delivered_ce	__u32	I	segments
200	tcpi_bytes_sent	__u64	Q	bytes
208	tcpi_bytes_retrans	__u64	Q	bytes
216	tcpi_dsack_dups	__u32	I	segments
220	tcpi_reord_seen	__u32	I	count
224	tcpi_rcv_ooopack	__u32	I	segments
228	tcpi_snd_wnd	__u32	I	bytes

* Bit fields, decoded as one byte each: `wscale` holds `tcpi_snd_wscale:4` and `tcpi_rcv_wscale:4`; `app_limited_flags` holds `tcpi_delivery_rate_app_limited:1` and `tcpi_fastopen_client_fail:2`.

Sampling and interval measurements. While sending, `send_and_sample` uses `select()` with a timeout so that sending never blocks past the next sample deadline. Sample k is due at `start + k * interval` (computed from k, so lateness does not accumulate), with a baseline sample at $t = 0$ and the final one clamped to the end of the test. Each sample's timestamp is taken with `time.monotonic()` immediately after `getsockopt`, so it matches the counters. Interval quantities use the difference between consecutive samples $i-1$ and i and the **measured** elapsed time, not the nominal interval:

$$\text{goodput}_i = \frac{\texttt{bytes_acked}_i - \texttt{bytes_acked}_{i-1}}{t_i - t_{i-1}} \cdot 8 \quad \text{[bits/s]}$$

$$\text{retrans_segs_interval}_i = \texttt{total_retrans}_i - \texttt{total_retrans}_{i-1} \quad \text{[segments]}$$

`tcpi_bytes_acked` counts bytes the receiver acknowledged, so goodput excludes data still sitting in our send buffer and counts retransmitted data only once.

Loss proxy. TCP never observes loss directly; it infers it and retransmits. We use **segments retransmitted per interval**, the change in the cumulative counter `tcpi_total_retrans`. The alternatives are snapshots that miss events between samples: `tcpi_retransmits` resets to 0 on every new ACK, and `tcpi_lost`

counts only the segments presumed lost at the instant of the sample. `total_retrans` includes spurious retransmissions (also logged as `tcpi_dsack_dups`), so the proxy can over-count loss on paths that reorder packets.

Stored format. `results/20261021-152852/samples.csv` has one row per sample with the run id, repetition, destination host, port and IP, requested and actually used algorithm, `t`, `unix_time`, `goodput_bps`, `retrans_segs_interval`, and every decoded `tcp_info` field. `metadata.json` maps every measured column to its unit and records the host kernel, the struct format, the selected destinations, and each run's outcome.

2(b) Visualization

The plots are generated automatically at the end of every campaign (`plot_tcp_stats`). The representative destination, `speed1.fiberby.dk:9213`, is picked by `choose_representative_destination`: among the destinations with data for every algorithm and no failed repetition, the one whose mean goodput is the median, so the plots show a typical path rather than the fastest or slowest. Each repetition is one line (time series) or one color (scatter).

Generated by `plot_time_series`. cwnd and RTT are snapshots at each sample time, drawn as lines; retransmissions and goodput describe a whole interval, drawn as steps.

Generated by `plot_scatter`. Each point is one sample. Goodput is the average over the interval ending at the sample, while cwnd and RTT are snapshots at its end, so a point taken right after a sudden change pairs the new cwnd with the previous interval's goodput.

2(c) Observations

Congestion window (`snd_cwnd`, segments). The largest number of unacknowledged segments the sender allows in flight. A window-limited sender delivers about one window per round trip, so in congestion avoidance

$$\text{goodput} \approx \frac{\min(\text{cwnd} \cdot \text{MSS},\ \text{rwnd})}{\text{RTT}}.$$

CUBIC grows the window as a cubic function of the time since the last loss, $W(t) = C(t-K)^3 + W_{max}$: it rises quickly, flattens near the window W_{max} where the last loss happened, then probes beyond it. On a loss it cuts the window to $\beta = 0.7$ of its value. We therefore expect goodput to follow cwnd (a roughly linear cwnd vs goodput scatter with slope MSS / RTT) as long as the window is below the bandwidth-delay product (BDP). Once cwnd exceeds the BDP, extra window only fills the bottleneck queue: goodput stops growing and RTT rises instead.

Smoothed RTT (`tcpi_rtt`, shown in ms) and minimum RTT. `srtt` is an exponentially weighted average of RTT samples (gain 1/8, RFC 6298), and `rttvar` tracks its variation. `min_rtt` approximates the propagation delay, so `srtt - min_rtt` estimates the queueing delay our own traffic and cross traffic add. For a loss-based algorithm like CUBIC we expect RTT to climb as the window grows past the BDP and the bottleneck buffer fills, while goodput plateaus at the bottleneck rate; RTT drops back after each window cut drains the queue. At a fixed window, higher RTT means lower goodput (the formula above), so the RTT vs goodput scatter should show goodput flat or falling as RTT grows, not rising.

Retransmitted segments per interval (loss proxy). Each loss event detected by duplicate ACKs, SACK, or RACK triggers fast retransmit and a multiplicative decrease, so we expect goodput to dip in the interval containing the loss or the next one. A retransmission timeout is worse: the window falls to 1 segment and slow start restarts, so goodput collapses for several intervals (also visible as nonzero `tcpi_backoff`). Intervals with retransmissions should therefore cluster at lower goodput in the scatter, although a single retransmission costs little if the window recovers within the same interval.

Goodput. Bytes acknowledged per second. It is bounded by the bottleneck link capacity, our share of it against cross traffic, the server's receive window (`tcpi_snd_wnd`), and any per-client rate limit on the server.

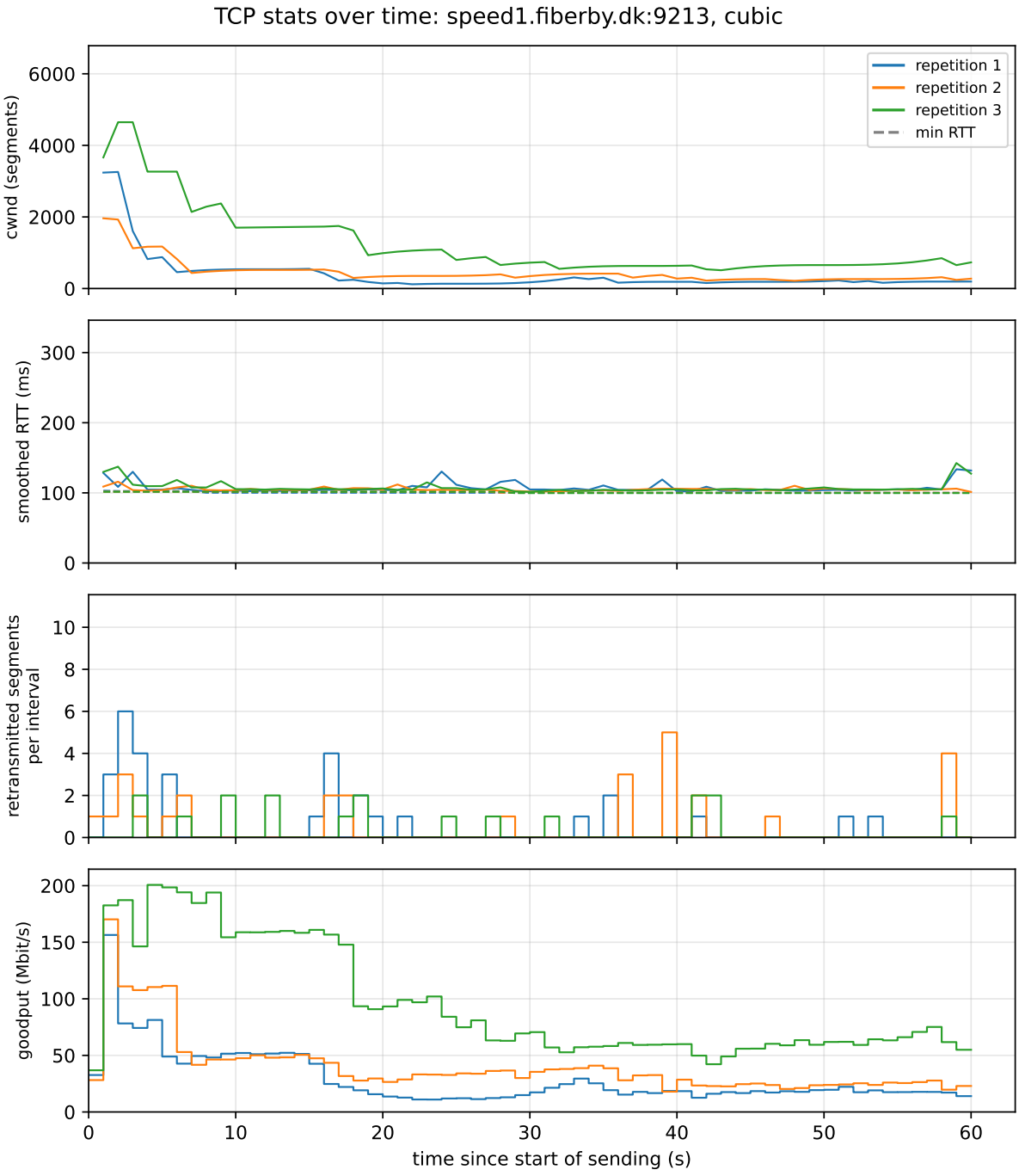


Figure 2: TCP stats over time (CUBIC). From top: congestion window, smoothed RTT (dashed: minimum RTT), retransmitted segments per interval, goodput. Generated by `report.plot_time_series`.

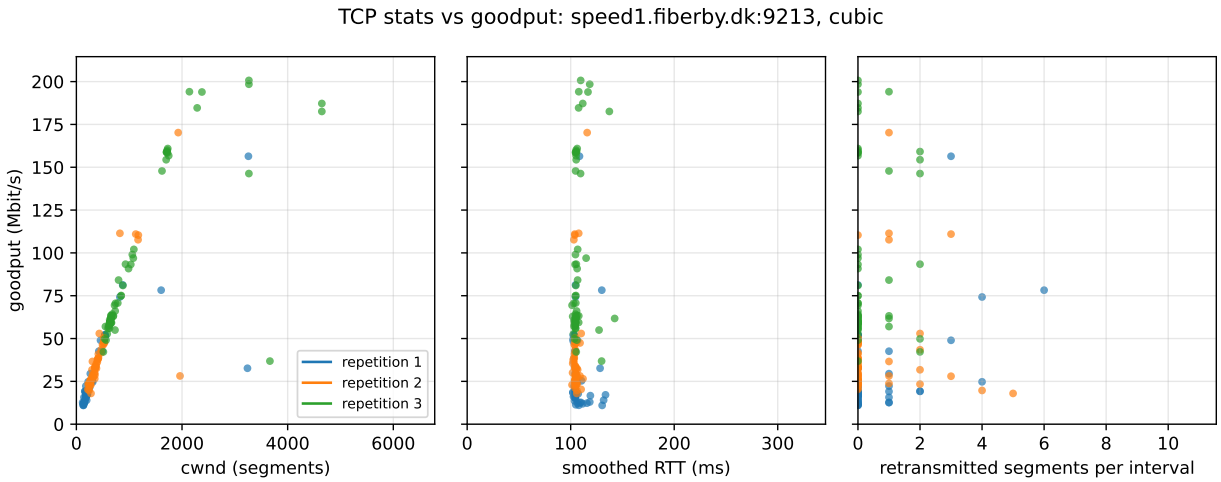


Figure 3: Congestion window, smoothed RTT, and retransmissions against goodput (CUBIC). Generated by `report.plot_scatter`.

Anomalous behavior

We checked all 30 cubic runs (10 destinations x 3 repetitions) in `samples.csv`, not only the representative destination. Five behaviors stand out.

1. In-flight data is capped at about 2.8 MB, so cwnd is often not the limit. No cubic run ever has more than 2,315 unacknowledged segments (`tcpi_unacked`), about 2.87 MB, and in 20 of the 30 runs `unacked` sits at 2,100 to 2,400 segments for 10 or more of the 60 samples, often while cwnd is far larger. On `123.53.44.221` (repetition 3), for example, cwnd sits at 4,630 segments for the whole minute but only about 2,224 are ever in flight. The cap is the same on every path, from 25 ms to 250 ms of RTT, so it is not set by the network. It is not the receiver either: at the cap the server advertises 3.1 to 16.8 MB (`snd_wnd`), always more than is in flight. At the cap, `notsent_bytes` has a median of 750 bytes and the kernel flags the delivery rate as application-limited in 95% of samples, meaning the socket had nothing more to send. Our send buffer was full of unacknowledged data: Linux's default maximum send buffer is 4 MiB (`net.ipv4.tcp_wmem`), and part of that is per-packet bookkeeping, which leaves roughly 2.8 MB for payload. (We did not record `tcp_wmem` on the measurement host; `sysctl net.ipv4.tcp_wmem` confirms it. The kernel's own `sndbuf_limited` counter stays at or below 6.5% of busy time, likely because our client waits in `select()` rather than blocking in `send()`, but we did not verify this.)

Two consequences follow:

- *Goodput is window-limited on long paths.* With 2.8 MB in flight per RTT, the ceiling is 2.8 MB · 8/RTT: about 91 Mbit/s at `123.53.44.221`'s 241 ms and 88 Mbit/s at `speedtestfl.telecom.mu`'s 250 ms. Their observed 95th percentiles are 88 and 81 Mbit/s for cubic, and almost the same for reno and bbr (Section 3(a)), so on these paths every algorithm hits our buffer, not the network.
- *cwnd stops at about twice the in-flight data.* Linux only grows cwnd while the flow is *cwnd-limited*; in slow start it allows growth while `cwnd < 2 * max_packets_out` (`tcp_is_cwnd_limited()` in `include/net/tcp.h`). With in-flight capped near 2,300 segments, cwnd freezes near 4,600: the runs that saw little early loss all peak at 4,550 to 4,760 segments. On `123.53.44.221` repetition 3 there was no loss at all (`total_retrans = 0`), so the window stayed at this value for the whole test and does not describe the path.

2. Large RTT inflation on short paths (bufferbloat). The same 2.8 MB in flight is far more than a short path's bandwidth-delay product, and the excess waits in a queue. On `speedtest.wdc2.us.leaseweb.net` (min RTT 26.4 ms), repetition 3, srtt is about 120 ms during steady sending and peaks at 13.6x `min_rtt`. This matches a simple queue estimate: at the observed ~185 Mbit/s, the BDP is 185 Mbit/s · 26.4 ms ≈ 0.61

MB, so about 2.2 MB is queued, adding 2.2 MB/23 MB/s ≈ 95 ms for a predicted srtt of about 121 ms. `la.speedtest.clouvider.net` and `speedtest6.cwpanama.net` show the same effect (median srtt 1.2 to 2.2x `min_rtt`, peaks 3.6 to 6.4x). The long paths, whose BDP is close to 2.8 MB, have median srtt within 4 to 11% of `min_rtt`. We cannot tell from the sender whether this queue is at the path's bottleneck or in the measurement host's own virtual network (WSL2); either way, the extra delay comes from our own data. CUBIC does not react to it, since it only backs off on loss.

3. Slow-start overshoot, then a slow decline on the representative path. On `speed1.fiberby.dk` (plots above) cwnd reaches 1,962 to 4,647 segments within the first 2 s. At its 100 ms RTT, 3,000 segments would be about 300 Mbit/s, more than the path delivers, so the first losses cut the window. After that, goodput falls steadily, from 85 to 150 Mbit/s in the first 5 s to 18 to 60 Mbit/s in the last 20 s. Here the flow *is* cwnd-limited: `unacked` equals cwnd and 2.5 MB waits unsent in the buffer. Yet the median srtt is within 5% of `min_rtt`, so there is no queue, and the retransmission rate is tiny (0.00 to 0.02% of segments). The 1 to 6 segment loss events every few seconds therefore do not come from our own queue overflowing; they look like random loss or cross traffic. Each one cuts cwnd to 70%, and at 100 ms RTT CUBIC regrows too slowly to get back before the next one, so the window ratchets down to roughly 150 to 750 segments. The cwnd vs goodput scatter is a straight line through the origin with slope close to MSS / RTT, as the formula above predicts: 1,000 segments · 1,238 B · 8 / 103 ms ≈ 96 Mbit/s, and the points at cwnd 1,000 sit near 95 to 100 Mbit/s.

4. Some repetitions collapse to 1 to 5 Mbit/s. Six cubic runs average under 5 Mbit/s on paths where another repetition of cubic, minutes earlier or later, averages 80 to 160 Mbit/s:

Destination	Repetition	Mean goodput (Mbit/s)	Retransmitted segments	Other cubic repetitions (Mbit/s)
`189.25.101.151`	1	2.8	3.7%	4.8, 99.4
`189.25.101.151`	2	4.8	2.1%	2.8, 99.4
`iperf.pendc.com`	2	1.3	2.1%	134.8, 93.6
`ping-ams1.online.net`	2	1.3	1.9%	149.6, 98.3
`speedtestfl.telecom.mu`	2	1.9	0.9%	43.9, 0.6
`speedtestfl.telecom.mu`	3	0.6	1.6%	43.9, 1.9

In these runs the median cwnd is 14 to 53 segments and `ssthresh` keeps getting cut, while srtt stays within 6 to 11% of `min_rtt`. So the path is losing 1 to 4% of our segments without any queue building up, which again points away from our own sending. Loss-based TCP cannot run fast at that loss rate: the Mathis model, rate $\approx \frac{\text{MSS}}{\text{RTT}} \cdot \frac{1.22}{\sqrt{p}}$, predicts 0.3 to 1.0 Mbit/s for these runs, the same order of magnitude as the observed 0.6 to 4.8 Mbit/s and far from the 80 to 160 Mbit/s of the loss-free repetitions. All four of these destinations list a port range (e.g. `5201-5209`), i.e. run several iperf3 servers on one host, so other clients' tests on neighboring ports may share the server's link; we cannot confirm the cause from the client side. On the same paths in the same repetition, bbr often kept much higher goodput at a similar loss rate (`iperf.pendc.com` repetition 2: 51 Mbit/s with 1.5% retransmitted), which Section 3(a) discusses.

5. Segments are 1,238 bytes instead of 1,448. Every cubic sample on every destination has `snd_mss = 1238`, although the path MTU is 1,500 (`pmtu`) and we advertise 1,448 (`advmss`). Since every server shows the same value, it is almost certainly a local MSS clamp on the measurement host's network (WSL2's virtual network or the Windows host), not the servers. It does not change the goodput calculation, which uses bytes, but it means a cwnd of N segments here is $N \cdot 1238$ bytes, about 15% less than a standard Ethernet segment, and each byte needs about 17% more packets and ACKs.

Retransmission timeouts. None were visible in the cubic samples: `backoff` is 0 and `ca_state` is never 4 (Loss) in any sample. Because these fields are snapshots, an RTO that recovered within one 1 s interval could still have been missed.

3. Compare Algorithms and Design a Congestion Control Algorithm

3(a) Compare CUBIC, Reno, and BBR

Same campaign as above, `results/20261021-152852`: the same 10 destinations, 60 s duration, 1.0 s sampling interval, and 3 repetitions, with `--algorithms cubic reno bbr` so every destination gets all three algorithms back to back within each repetition (test order: for each repetition, for each destination, every algorithm). The algorithm is set on the data socket before `connect()` and the actually-applied algorithm read back afterward, both in `open_data_stream` (`setsockopt(IPPROTO_TCP, TCP_CONGESTION, ...)` before connecting, then a matching `getsockopt` readback). The kernel-reported algorithm actually used (`algorithm_used` in `samples.csv`) matched the requested one on every completed run. Host kernel: `6.18.54.1-microsoft-standard-WSL2` (the stock WSL2 kernel ships `tcp_bbr.ko`, loadable without root, once WSL is updated past the 5.15 line; see "Reproducing the results").

Out of 90 scheduled runs (10 destinations x 3 algorithms x 3 repetitions), 89 completed. The bbr run on `la.speedtest.clouvider.net` (port 5205, repetition 2) failed all 3 retry attempts with the server rejecting parameter exchange (`server busy / ACCESS_DENIED`) and was skipped, per the bounded-retry robustness requirement. No destination is missing data entirely (`destinations_without_data: []` in `metadata.json`).

Representative destination for the plots below: same as Part 2, `speed1.fiberby.dk:9213`.

Goodput summary, all destinations (Mbit/s), from `results/20261021-152852/goodput_summary.csv`:

host	port	algorithm	samples	min	median	mean	p95
123.53.44.221	5215	bbr	180	0.51	27.38	38.61	81.51
123.53.44.221	5215	cubic	180	0.63	83.17	80.31	88.10
123.53.44.221	5215	reno	180	1.22	89.11	85.88	91.96
156.146.40.65	5201	bbr	180	8.64	162.69	153.83	175.33
156.146.40.65	5201	cubic	180	3.77	163.04	135.89	175.85
156.146.40.65	5201	reno	180	12.79	161.70	152.46	174.58
189.25.101.151	30007	bbr	180	0.84	95.89	85.13	105.61
189.25.101.151	30007	cubic	180	0.19	2.44	35.68	106.07
189.25.101.151	30007	reno	180	0.01	51.98	42.63	96.87
iperf.pendc.com	5208	bbr	180	1.91	99.35	94.86	149.53
iperf.pendc.com	5208	cubic	180	0.51	94.50	76.56	141.76
iperf.pendc.com	5208	reno	180	0.35	20.51	51.16	131.28
la.speedtest.clouvider.net	5205	bbr	120*	35.39	129.51	140.17	216.92
la.speedtest.clouvider.net	5205	cubic	180	8.87	145.66	140.11	215.65
la.speedtest.clouvider.net	5205	reno	180	3.30	117.89	107.05	191.65
ping-ams1.online.net	5209	bbr	180	18.57	165.37	149.58	195.23
ping-ams1.online.net	5209	cubic	180	0.47	93.94	83.05	188.86
ping-ams1.online.net	5209	reno	180	21.23	121.19	122.44	171.63
speed1.fiberby.dk	9213	bbr	180	21.88	147.54	139.45	196.68
speed1.fiberby.dk	9213	cubic	180	10.97	41.30	54.80	160.03
speed1.fiberby.dk	9213	reno	180	3.67	22.28	47.42	198.34
speedtest.wdc2.us.leaseweb.net	5205	bbr	180	71.06	194.45	187.35	227.14
speedtest.wdc2.us.leaseweb.net	5205	cubic	180	31.22	191.05	175.85	224.78
speedtest.wdc2.us.leaseweb.net	5205	reno	180	2.54	182.14	128.50	223.89
speedtest6.cwpanama.net	5508	bbr	180	52.06	135.93	141.64	211.57
speedtest6.cwpanama.net	5508	cubic	180	33.78	162.37	163.47	224.56
speedtest6.cwpanama.net	5508	reno	180	0.89	98.88	76.96	138.59
speedtestfl.telecom.mu	5209	bbr	180	0.49	49.51	52.26	81.07
speedtestfl.telecom.mu	5209	cubic	180	0.28	1.41	15.48	80.71
speedtestfl.telecom.mu	5209	reno	180	0.22	6.96	20.07	79.06

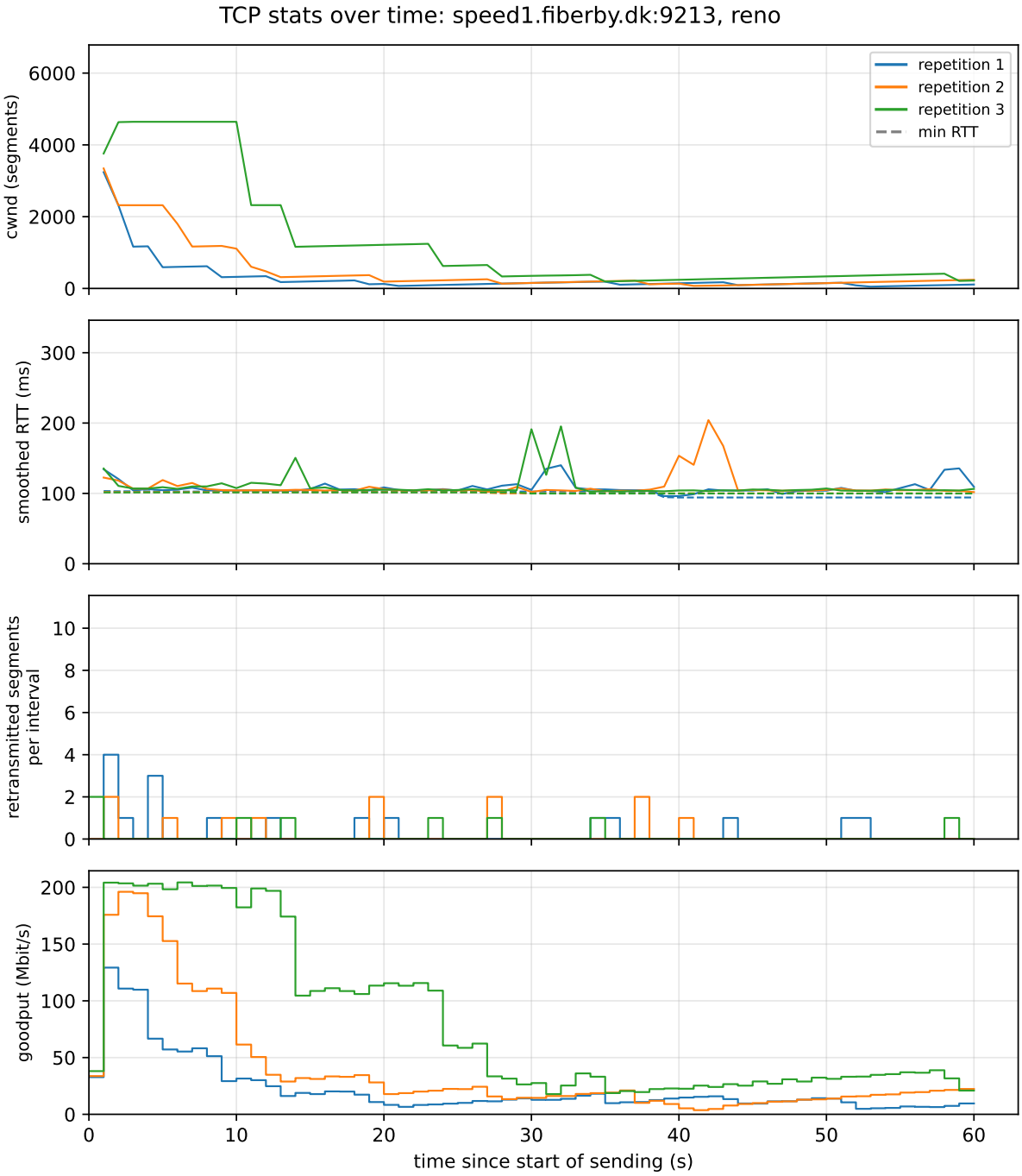


Figure 4: TCP stats over time (Reno).

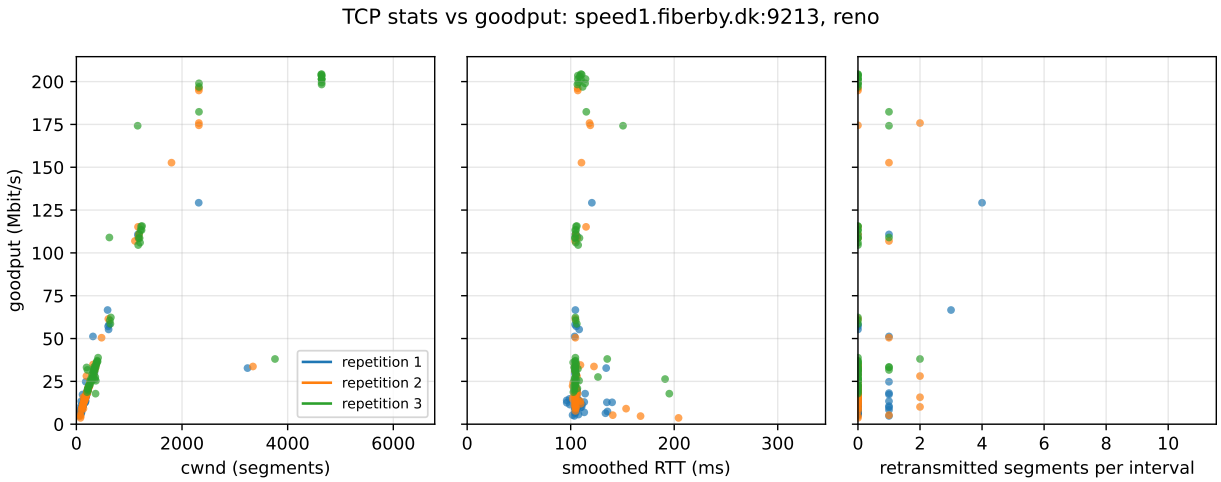


Figure 5: Congestion window, smoothed RTT, and retransmissions against goodput (Reno).

* 120 samples (2 of 3 repetitions) because the third repetition's run on this destination/algorithm was skipped after 3 failed attempts (see above).

Results by algorithm

All goodput values are in Mbit/s. "srtt / min RTT" is each sample's smoothed RTT divided by the path's minimum RTT, so 1.0 means no queueing delay and 2.0 means the queue doubled the RTT. "Retransmitted" is retransmitted segments over all data segments sent (our loss proxy). Computed from `samples.csv` (all destinations, all repetitions pooled).

Algorithm	Runs	Median goodput	Mean goodput	Worst run (mean)	Runs under 10 Mbit/s	Median srtt / min RTT	p95 srtt / min RTT	Retransmitted
cubic	30	92.1	96.1	0.6	6	1.09	3.07	0.029%
reno	30	88.7	83.5	0.5	5	1.09	3.17	0.039%
bbr	29	119.3	117.5	20.4	0	1.11	3.09	0.126%

Per destination, sorted by minimum RTT. Each algorithm cell is *median goodput (Mbit/s) / median srtt (ms) / retransmitted / CoV*, where CoV is the coefficient of variation (standard deviation / mean) of the three repetitions' mean goodput, our measure of run-to-run variability. The best median goodput per row is in bold. "Ceiling" is the most our client can deliver on that path given the ~2.8 MB in-flight limit set by our send buffer (Section 2(c), finding 1): 2.8 MB · 8/min RTT.

Destination	min RTT (ms)	Ceiling (Mbit/s)	cubic	reno	bbr
`speedtest.wdc2.us.leaseweb.net`	26	877	191 / 65 / 0.03% / 0.04	182 / 66 / 0.04% / 0.67	**194 / 51 / 0.02% / 0.04**
`la.speedtest.clouvider.net`	55	409	**146 / 114 / 0.02% / 0.29**	118 / 106 / 0.02% / 0.29	130 / 133 / 0.01% / 0.24
`speedtest6.cwpanama.net`	67	334	**162 / 96 / 0.01% / 0.24**	99 / 78 / 0.04% / 0.69	136 / 138 / 0.00% / 0.16

Destination	min RTT (ms)	Ceiling (Mbit/s)	cubic	reno	bbr
`ping-ams1.online.net`	92	243	94 / 116 / 0.07% / 0.74	121 / 156 / 0.08% / 0.09	**165 / 119 / 0.10% / 0.15**
`speed1.fiberby.dk`	94	238	41 / 105 / 0.01% / 0.53	22 / 105 / 0.00% / 0.52	**148 / 108 / 0.01% / 0.10**
`156.146.40.65`	118	189	**163 / 129 / 0.01% / 0.24**	162 / 134 / 0.00% / 0.06	163 / 131 / 0.00% / 0.02
`iperf.pendc.com`	141	159	95 / 160 / 0.01% / 0.73	21 / 163 / 0.07% / 0.74	**99 / 157 / 0.32% / 0.33**
`189.25.101.151`	204	110	2 / 218 / 0.19% / 1.26	52 / 222 / 0.07% / 0.74	**96 / 216 / 0.18% / 0.11**
`123.53.44.221`	234	96	83 / 256 / 0.01% / 0.03	**89 / 248 / 0.00% / 0.01**	27 / 272 / 0.68% / 0.63
`speedtestfl.telecom.mu`	250	90	1 / 289 / 0.21% / 1.30	7 / 267 / 0.23% / 0.59	**50 / 279 / 0.93% / 0.21**

BBR has the best median goodput on 6 of 10 destinations, the best median and mean overall, and is the only algorithm with no collapsed run: its worst run averaged 20.4 Mbit/s, against 0.6 for cubic and 0.5 for reno. It also retransmits about four times as much (0.126% vs 0.03 to 0.04%). Median RTT inflation is almost the same for all three (1.09 to 1.11x). The differences come from four operating regimes, described next.

Operating regimes

Two facts from Section 2(c) frame everything below. First, a window-based sender delivers at most one window per RTT, so goodput $\approx \min(\text{cwnd} \cdot \text{MSS},\ \text{rwnd},\ \text{send buffer})/\text{RTT}$. Second, on these runs the send buffer allows only ~2.8 MB in flight. A path's bandwidth-delay product (BDP) is the in-flight data needed to fill it; whenever the BDP exceeds 2.8 MB, the flow is buffer-limited no matter which algorithm runs, and whenever it is smaller, the excess sits in a queue.

1. Non-congestive loss on long paths: BBR wins by 1.4x to 48x. On `speed1.fiberby.dk`, `ping-ams1.online.net`, `189.25.101.151`, and `speedtestfl.telecom.mu`, bbr's median goodput is 148, 165, 96, and 50 Mbit/s, against 41, 94, 2, and 1 for cubic and 22, 121, 52, and 7 for reno. On three of these paths the median srtt is within 4 to 9% of the minimum RTT (on `ping-ams1.online.net`, 1.2 to 1.6x), so there is little or no standing queue, yet segments are still lost. That loss is not a signal that *our* flow overfilled a buffer; it comes from cross traffic or random drops. CUBIC and Reno treat every loss as congestion and cut cwnd (to 70% and 50%), then regrow it slowly: Reno adds one segment per RTT, so regaining 1,000 segments at 100 ms RTT takes 100 s, longer than the whole test. The representative destination shows this directly (time series in Section 2(b) and below): cubic and reno climb to 130 to 200 Mbit/s in the first seconds, and each sparse loss event ratchets the window down until goodput settles at 10 to 60 Mbit/s, with RTT flat at the minimum the whole time. BBR does not use loss as its main signal. It estimates the bottleneck rate as the maximum delivery rate seen over the last ~10 RTTs and the propagation delay as the minimum RTT over the last 10 s, and paces at that rate with about one BDP in flight. Isolated losses barely move either estimate, so on the same path bbr holds 100 to 200 Mbit/s for most of the 60 s, with a scattered cwnd but no downward trend. The worst cases for the loss-based algorithms are the collapsed runs of Section 2(c), finding 4: at 1 to 4% loss and 200+ ms RTT, the Mathis model limits CUBIC and Reno to about 1 Mbit/s, while bbr in the same repetition on the same paths reached 40 to 118 Mbit/s.

2. Paths at the buffer ceiling: all three tie. On `156.146.40.65` (BDP near the 2.8 MB limit) the medians are 163, 162, and 163 Mbit/s, and on `123.53.44.221` cubic and reno reach 83 and 89 Mbit/s against a 96 Mbit/s ceiling. Here none of the algorithms' window rules matter: each is held at the same ~2,250 segments in flight by our send buffer, the path is clean (0.00 to 0.01% retransmitted), and goodput is set by

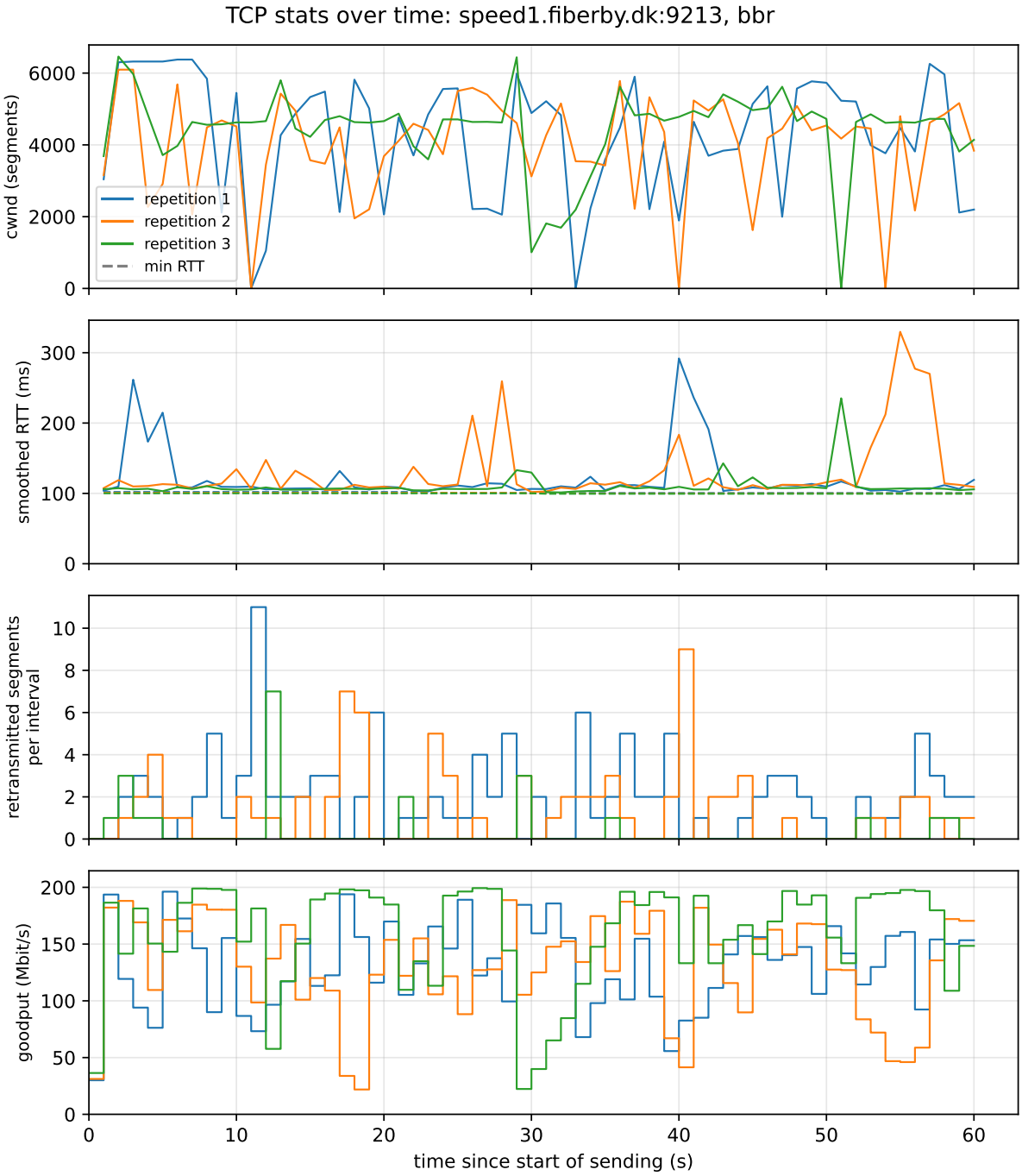


Figure 6: TCP stats over time (BBR).

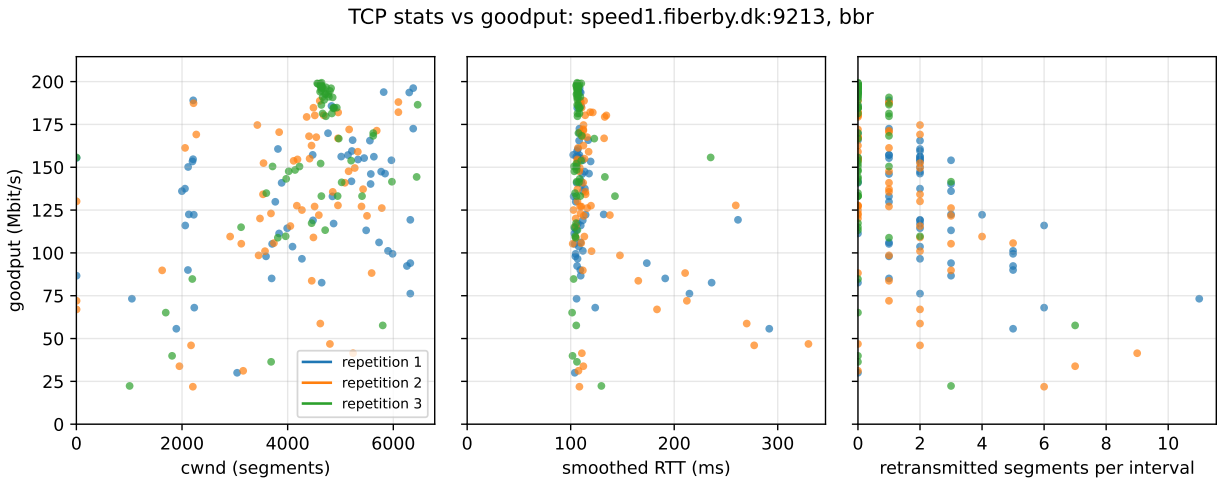


Figure 7: Congestion window, smoothed RTT, and retransmissions against goodput (BBR).

2.8 MB per RTT. These paths say nothing about which algorithm is better, only that a larger send buffer (`net.ipv4.tcp_wmem`) would be needed to compare them; the BBR exception on `123.53.44.221` is regime 4.

3. Short paths (BDP well below 2.8 MB): CUBIC wins on throughput, and every algorithm builds a queue. On `speedtest.wdc2.us.leaseweb.net`, `la.speedtest.clouvider.net`, and `speedtest6.cwpanama.net` (26 to 67 ms, BDP 0.6 to 1.4 MB at the observed rates), the ceiling is 334 to 877 Mbit/s, far above the 100 to 200 Mbit/s achieved, so these flows are limited by the network. CUBIC has the best median on two of them (146 and 162 Mbit/s) and is within 2% of bbr on the third. Its cubic growth function quickly returns to the window where the last loss happened, which suits a clean, short path where losses are rare and the window needed is small. Reno is worst on all three, and its 0.67 to 0.69 CoV on two of them comes from a single bad run each (6.2 and 3.0 Mbit/s with 1.5 to 1.7% retransmitted, between good cubic and bbr runs minutes apart): after heavy loss, one segment per RTT is too slow to recover. In RTT, the three are similar: srtt is 1.4 to 2.5x the minimum for every algorithm, because the send buffer lets all of them put 2.1 to 2.8 MB in flight (median) on a path that needs at most 1.4 MB, and the rest queues (Section 2(c), finding 2). BBR is meant to avoid this queue by keeping in-flight near one BDP, and it does have the lowest RTT on the shortest path (51 vs 65 ms). On the other two, however, its srtt is the highest (133 and 138 ms vs 114 and 96 ms for cubic): its cwnd is 2x its BDP estimate (median 3,400 to 4,300 segments here), so like the others it is held by the send buffer rather than by its own model, and its bandwidth probing (pacing 25% above the estimate in one of every eight phases) adds to the queue.

4. Shallow buffer on a long path: BBR loses. On `123.53.44.221` (234 ms), bbr averaged 20.4 and 22.6 Mbit/s in repetitions 1 and 2, while cubic and reno, run just before it on the same path, got 78 to 87 Mbit/s with almost no loss. In those two bbr runs 1.6% of segments were retransmitted and 84 to 89% of samples are in loss recovery (`ca_state = 3`). BBR's Startup phase paces at about 2.9x its estimate to find the bottleneck quickly; here the pacing rate reached 227 Mbit/s against 15 to 40 Mbit/s actually delivered, so it overflowed what appears to be a shallow buffer. Because BBR (v1) does not slow down on loss, the overshoot keeps causing loss, the delivery-rate samples taken during recovery stay low (15 to 40 Mbit/s), and its bandwidth estimate and pacing rate stay low with them. In repetition 3 the Startup loss burst (455 segments) ended quickly, bbr reached 72.8 Mbit/s at the buffer ceiling, and retransmitted nothing for the rest of the test. So on this path bbr's outcome depended on how its startup went, which is why its CoV there is 0.63 against 0.01 to 0.03 for the loss-based algorithms. High retransmission rates with shallow buffers are a known weakness of BBRv1 (Hock, Bless, and Zitterbart, "Experimental Evaluation of BBR Congestion Control", ICNP 2017).

Reading the BBR and Reno plots. In bbr's time series the cwnd drops to near 0 a few times per run (at 11 and 33 s in repetition 1, at 11, 40, and 54 s in repetition 2, at 51 s in repetition 3). This is BBR's

ProbeRTT phase: if the minimum RTT has not been refreshed for 10 s, BBR cuts in-flight to 4 segments for about 200 ms to drain the queue and re-measure the propagation delay. It lasts only a fraction of a 1 s sample, so goodput barely dips, and a 1 s sampler catches only the ProbeRTT phases that happen to overlap a sample. Separately, the reported cwnd is only meaningful when it is the limit: on `ping-ams1.online.net` reno's cwnd grows to 225,827 segments (and the derived pacing rate to 26 Gbit/s) while only ~2,250 segments are ever in flight, because the send buffer, not cwnd, was binding (Section 2(c), finding 1).

Summary of regimes

- **Lossy long path, little or no queueing:** BBR is far better in throughput (1.4x to 48x). Loss-based algorithms misread random loss as congestion and recover too slowly at high RTT.
- **Clean path whose BDP exceeds the send buffer:** no difference; every algorithm is buffer-limited.
- **Clean short path (small BDP):** CUBIC gives the best throughput; Reno is the slowest to recover from a bad loss episode. All three inflate RTT here because of our send buffer; BBR had the lowest RTT only on the shortest path.
- **Shallow buffer on a long path:** BBR can lose badly, because its probing causes loss that it does not back off from.

In RTT, the algorithms differ much less than in throughput: across all samples the median inflation is 1.09 to 1.11x for all three and the 95th percentile 3.07 to 3.17x. In our setup, queueing delay was set mainly by our send buffer relative to the path's BDP, not by the algorithm.

Variability across runs

Run-to-run variability is large. The same destination and algorithm can differ by 100x between repetitions (cubic on `speedtestfl.telecom.mu`: 43.9, 1.9, and 0.6 Mbit/s), and 12 of the 30 destination/algorithm groups have a CoV above 0.5 (5 cubic, 6 reno, 1 bbr). Because the three algorithms run back to back within a repetition, about 70 s apart, a sudden change between neighboring runs points to the network rather than the algorithm: reno's 6.2 Mbit/s run on `speedtest.wdc2.us.leaseweb.net` (repetition 1) falls between cubic at 172 and bbr at 199. BBR is the most consistent algorithm on the lossy paths (CoV 0.10 to 0.21 on `speed1.fiberby.dk`, `189.25.101.151`, and `speedtestfl.telecom.mu`, against 0.52 to 1.30 for cubic and reno), and the least consistent where its startup can overshoot (`123.53.44.221`, 0.63). With only three repetitions per group, a single bad run moves the mean a lot, which is why we compare medians of the pooled 1 s samples and treat differences under about 10% (for example `156.146.40.65`, or bbr vs cubic on `speedtest.wdc2.us.leaseweb.net`) as ties.

Limitations of comparing over public Internet paths

- **The algorithms never see the same network.** Tests on one path run one after another, not at the same time, and cross traffic, routing, and the server's other clients change between them. Hosts that list a port range run several iperf3 servers sharing one link, so other users' tests may overlap ours. We reduce this with back-to-back order and repetitions, but cannot remove it.
- **Few samples.** Three repetitions of 60 s per group is enough to see the large effects above, not to estimate small differences with confidence.
- **Our own host limits the comparison.** The ~2.8 MB send-buffer limit removes the high-BDP regime entirely (every algorithm is capped there) and causes most of the queueing we measured on short paths. The measurement host ran WSL2, whose virtual network clamps MSS to 1,238 bytes and may add its own queue. A follow-up run should raise `net.ipv4.tcp_wmem` and use a native Linux host.
- **Single flow, unknown bottleneck.** Each test is one flow, so we cannot observe fairness between algorithms, and from the sender we cannot tell where the bottleneck is, how deep its buffer is, or whether a server enforces a rate limit.
- **Sampling.** `TCP_INFO` snapshots every 1 s miss events shorter than a sample (ProbeRTT, short timeouts), and 60 s is only a few CUBIC epochs on a 250 ms path.
- **Selection bias.** Destinations were the first servers to answer a probe, not a representative sample of Internet paths, and one bbr run (`la.speedtest.clouvider.net`, repetition 2) is missing.

3(b) Design a hand-written algorithm

We call our design **Delay-Gated Loss (DGL)**. It is a window-based AIMD algorithm like Reno, with two changes driven by our measurements: a loss only triggers a large window cut when the RTT shows a queue (real congestion), and the window never falls below what the path has recently proven it can deliver. Assignment 3 implements it as an eBPF `struct_ops` program registered as `cs422cc`.

What the measurements told us

- **Loss without queueing is common and loss-based algorithms overreact to it.** On `speed1.fiberby.dk`, `189.25.101.151`, and `speedtestfl.telecom.mu`, segments were lost while the median srtt stayed within 4 to 9% of the minimum RTT (Section 3(a), regime 1). CUBIC and Reno cut the window on every loss and their medians fell to 1 to 52 Mbit/s, while BBR, which mostly ignores loss, kept 50 to 148 Mbit/s. A loss with no queue is weak evidence of congestion.
- **When we do cause congestion, the RTT shows it first.** On the short paths every algorithm built a queue that raised srtt to 1.4 to 2.5x the minimum (regime 3), long before most losses. RTT inflation is an early, reliable congestion signal there.
- **Overshooting at startup is expensive.** CUBIC's slow start reached 2,000 to 4,600 segments in 2 s on `speed1.fiberby.dk`, and BBR's 2.9x startup gain caused persistent loss on `123.53.44.221` (regime 4). Leaving slow start when the queue starts to grow avoids both.
- **Delivered throughput is robust to isolated losses.** BBR's success came from basing its rate on the measured delivery rate, which a single lost segment barely changes. We reuse that idea only as a floor, not as the main control loop, to keep the algorithm simple.

Signals

DGL uses three inputs, all available to a Linux congestion control module:

Input	How it is computed	Used for
RTT	`srtt` from the kernel; `min_rtt` = minimum RTT sample over the last 10 s; `queue_delay = srtt - min_rtt`	deciding whether a queue exists (grow, hold, back off) and classifying each loss
Loss proxy	retransmitted segments per round trip (the same `total_retrans` difference we log in Part 2), divided by segments delivered in that round	catching heavy loss without queueing (e.g. a rate policer)
Throughput	delivery rate per round trip, `delivered bytes / round duration`, kept as a slowly decaying maximum `bw_max`	the window floor `bw_max * min_rtt` after a random loss

A *round* is one round trip: it ends when the data that was outstanding at its start has been acknowledged. Thresholds are fractions of `min_rtt` rather than fixed milliseconds, because our paths range from 26 to 250 ms.

Pseudocode

```
constants
    GROW_FRACTION       = 1/8     # grow only while queue_delay < min_rtt / 8
    CONGESTED_FRACTION  = 1/4     # queue_delay > min_rtt / 4 means congestion
    CONGESTION_BETA     = 1/2     # window multiplier after a congestion loss (Reno)
    RANDOM_LOSS_BETA    = 7/8     # window multiplier after a loss with no queue
```

```
    DELAY_BACKOFF       = 1/8     # window reduction per round while congested
    LOSS_RATE_LIMIT     = 5%      # per-round loss rate treated as congestion anyway
    BW_DECAY            = 15/16   # per-round decay of the bandwidth maximum
    MIN_RTT_WINDOW      = 10 s    # forget min_rtt after this long without a new minimum
    MIN_CWND            = 2       # segments

state (per connection)
    cwnd, ssthresh                    # segments; initial cwnd = 10, ssthresh = infinity
    min_rtt, min_rtt_time             # initially the first RTT sample
    bw_max = 0                        # bytes/s
    round_end, round_start_delivered, round_start_time, round_start_retrans
    last_round_loss_rate = 0
    backed_off_this_round = false

on every ACK (rtt_sample, acked_segments):
    # 1. RTT signal
    if rtt_sample < min_rtt or now - min_rtt_time > MIN_RTT_WINDOW:
        min_rtt = rtt_sample
        min_rtt_time = now
    queue_delay = srtt - min_rtt

    # 2. Throughput and loss proxy, once per round
    if delivered >= round_end:
        round_segments = delivered - round_start_delivered
        rate = round_segments * MSS / (now - round_start_time)
        bw_max = max(rate, bw_max * BW_DECAY)
        last_round_loss_rate = (total_retrans - round_start_retrans) / round_segments
        start a new round (round_end = delivered + segments in flight)
        backed_off_this_round = false

    # 3. Window update
    if cwnd < ssthresh:                                  # slow start
        if queue_delay > min_rtt * GROW_FRACTION:
            ssthresh = cwnd                              # queue forming: stop doubling
        else:
            cwnd = cwnd + acked_segments                 # doubles every RTT
    else if queue_delay < min_rtt * GROW_FRACTION:       # no queue: probe
        cwnd = cwnd + acked_segments / cwnd              # +1 segment per RTT
    else if queue_delay > min_rtt * CONGESTED_FRACTION and not backed_off_this_round:
        cwnd = max(cwnd * (1 - DELAY_BACKOFF), MIN_CWND)  # queue too long: drain it
        backed_off_this_round = true
    # otherwise (small queue): hold cwnd steady

on loss detected (fast retransmit or timeout), returns the new ssthresh:
    if queue_delay > min_rtt * CONGESTED_FRACTION or last_round_loss_rate > LOSS_RATE_LIMIT:
        # congestion: back off like Reno
        return max(cwnd * CONGESTION_BETA, MIN_CWND)
    else:
        # random loss: small cut, never below the recently measured BDP
        bdp_floor = bw_max * min_rtt / MSS
        return max(cwnd * RANDOM_LOSS_BETA, bdp_floor, MIN_CWND)
    # After a fast retransmit the kernel sets cwnd to this ssthresh; after a
    # timeout it sets cwnd = 1 and slow-starts back up to it.
```

```
on spurious loss detected (undo):
    cwnd = max(cwnd, cwnd before the reduction)
```

All fractions are powers of two or simple ratios (1/8, 1/4, 7/8, 15/16), so the eBPF implementation can use integer shifts and multiplications (the kernel has no floating point). The hooks map directly onto Linux's `tcp_congestion_ops`: the ACK path is `pkts_acked` (RTT sample) plus `cong_avoid` (window growth), the loss path is `ssthresh`, and the undo path is `undo_cwnd`.

Design choices

- **Reno's additive increase, not CUBIC's.** +1 segment per RTT is the simplest rule with a known fairness proof (below). CUBIC grows faster at high RTT, but DGL avoids most of the large cuts that make fast regrowth necessary, so the simpler rule is enough.
- **Classify loss by queueing delay.** If `queue_delay` exceeds a quarter of the base RTT when a loss happens, the bottleneck buffer is filling and the loss is likely ours: halve, like Reno. If there is essentially no queue, the loss is likely random or caused by unrelated traffic: cut by only 1/8. The 1/4 gate sits between the 1.04 to 1.09x srtt ratios on our lossy-but-unqueued paths and the 1.4 to 2.5x ratios on the paths where our own data queued.
- **Throughput floor after random loss.** `bw_max * min_rtt` is the in-flight data the path recently delivered at full rate. Never going below it after a random loss stops the downward ratchet of Section 2(c), finding 3: on `speed1.fiberby.dk` a loss every few seconds drove CUBIC from 1,000+ segments to 150 to 750, even though the path kept delivering at the higher rate. The decay (15/16 per round) lets the floor follow a real drop in capacity within about 10 to 20 round trips.
- **Back off on delay before loss.** Cutting by 1/8 per round while the queue exceeds a quarter of the base RTT keeps the RTT close to the minimum. On our short paths this would replace the 1.4 to 2.5x RTT inflation with at most about 1.25x, at a small cost in throughput.
- **Leave slow start when a queue appears.** Exiting as soon as `queue_delay` exceeds `min_rtt / 8`, similar to CUBIC's HyStart, avoids the multi-thousand-segment overshoots we measured.
- **A loss-rate guard.** A rate policer drops packets without building a queue, so the delay gate alone would call that loss random and keep sending too fast. A per-round loss rate above 5% is treated as congestion regardless of delay. 5% is above every per-run loss rate we measured (at most 3.7%), so random loss on our paths would not trigger it.

Expected behavior on our measured regimes

Regime (Section 3(a))	What DGL does	Expected result
1. Loss without queueing, long RTT	Cuts by 1/8, floored at the measured BDP	Holds close to the delivered rate, like BBR, instead of collapsing
2. BDP above our send buffer	Same as the others; the send buffer binds	Tie
3. Short path, our data queues	Delay backoff keeps the queue under `min_rtt / 4`; losses there count as congestion (halve)	Similar throughput, much lower RTT than CUBIC/Reno/BBR in our data
4. Shallow buffer, long RTT	Leaves slow start early; heavy loss trips the loss-rate guard	Avoids BBR's persistent-loss failure

Fairness and convergence

DGL flows sharing a bottleneck. Flows behind the same bottleneck see the same queue, so they classify losses the same way and back off at the same time. While the queue is short they all add one segment per RTT; when it grows past the threshold they all cut by the same *fraction* (1/8 per round, or 1/2 on a congestion loss). This is additive increase, multiplicative decrease: each cycle the absolute gap between two flows' windows stays the same during increase and shrinks during decrease, so the windows converge to equal shares (Chiu and Jain, 1989). The hold zone between `min_rtt / 8` and `min_rtt / 4` alone would not

equalize shares, because flows simply stop changing there; it is the AI/MD cycle around it that does. The sending rates converge to a stable operating point: total in-flight data near the path's BDP plus a small queue of between 1/8 and 1/4 of the base RTT, oscillating in a small sawtooth rather than filling the buffer.

Different RTTs. As with Reno, a flow adds one segment per *its own* RTT, so a flow with half the RTT grows twice as fast and gets a larger share. DGL is not RTT-fair.

Against CUBIC or Reno. DGL is expected to lose. Loss-based flows keep the bottleneck queue full, so DGL sees a permanently long queue, keeps backing off on delay, and is squeezed toward a small share. This is the classic problem of delay-based algorithms such as TCP Vegas. A fix would be to detect that the queue stays long despite our own backoff, assume a loss-based competitor, and switch to pure Reno behavior until the queue drains; we leave this out to keep the design simple.

The floor and newcomers. When a second flow joins, the bottleneck queue grows, so losses are classified as congestion and the throughput floor does not apply. A flow cannot use the floor to defend its old rate against a new competitor, so the floor should not hurt convergence. It could delay convergence on a path where a competitor causes loss without a visible queue, until `bw_max` decays.

Assumptions

- A single bottleneck whose buffer holds at least about `min_rtt / 4` of queueing delay, so congestion shows up in the RTT before it causes loss.
- The RTT is not too noisy: delay variation from wireless links, delayed ACKs, or ACK compression is small compared with `min_rtt / 8`.
- The queue drains at least once every 10 s, so `min_rtt` stays close to the true propagation delay. DGL's own delay backoff helps with this.
- Random loss is independent of our sending rate. If it is not (e.g. a policer), the loss-rate guard is the fallback.

Limitations

- Delay-based backoff makes DGL weak against loss-based flows (above).
- A shallow buffer, smaller than `min_rtt / 4` of queueing, produces loss before any queue is visible, so DGL would classify real congestion loss as random and cut by only 1/8. It would then be too aggressive toward other flows until the loss rate exceeded 5%.
- Queueing on the reverse (ACK) path also raises srtt, so DGL would back off although the forward path is not congested.
- A route change to a longer path looks like a queue until `min_rtt` expires after 10 s; DGL backs off needlessly during that time.
- The thresholds (1/8, 1/4, 7/8, 5%) are chosen from a small set of public paths and have not been tuned or tested against competing traffic.
- Under our measurement setup, the ~2.8 MB send-buffer limit (Section 2(c)) would cap DGL like the other algorithms on high-BDP paths.

Reproducing the results

Host kernel for `results/20261021-152852/`: `6.18.54.1-microsoft-standard-WSL2` (WSL2, Ubuntu 22.04 userspace), with `reno`, `cubic`, and `bbr` all available (`net.ipv4.tcp_available_congestion_control`). On this host `bbr` needed no `sudo modprobe tcp_bbr`: it loads automatically once WSL itself is updated past the kernel's 5.15 line (`wsl --update` from an elevated Windows terminal, then `wsl --shutdown` to actually restart the VM onto the new kernel). On a from-scratch Linux host, `sudo modprobe tcp_bbr` is the expected step the README documents.

Exact command line (from `metadata.json` `arguments`):

```
.venv/bin/python run_experiments.py listed_iperf3_servers.csv \
    -n 10 --duration 60 --interval 1 --repetitions 3 \
    --algorithms cubic reno bbr
```

Docker. The repository's Dockerfile (`FROM ubuntu:24.04`) packages the full pipeline: its default command runs the same campaign as above, from destination selection through the summary table and plots. To run it on a Linux or WSL2 host:

```
./run_wsl_docker.sh
```

The launcher loads `tcp_bbr` on the host if necessary, rejects Docker Desktop, builds the image, verifies all requested algorithms inside the container, and runs it with `--network host`, `--cap-add NET_ADMIN`, and a persistent results volume. With native Docker Engine, host networking gives the container the Linux/WSL2 host's network stack directly, with no NAT or proxy in between; the capability allows selecting algorithms outside `tcp_allowed_congestion_control` (usually bbr). Arguments after the launcher replace the default command, with the same options as `run_experiments.py`.

The campaign in `results/20261021-152852` was run with the same code directly on the WSL2 host (`.venv/bin/python run_experiments.py`, command above) rather than inside the container. On native Docker Engine with `--network host`, the corresponding sockets use the same host kernel, network stack, and path, so containerization changes only the standardized user space (Python and matplotlib), not the kernel measurements. This equivalence does **not** apply to Docker Desktop's proxied network path. `metadata.json` records the host kernel either way, since `platform.release()` reports the host kernel inside a container too.

Do not collect assignment measurements with Docker Desktop. In our validation, both Desktop's host and bridge modes proxied outbound TCP: a container reported a 0.647 ms minimum RTT to a server for which direct WSL2 measured 109.931 ms. The container was measuring the local proxy connection, not the Internet path. The automated `run_wsl_docker.sh` launcher therefore refuses measurement runs when `docker info` identifies Docker Desktop. Use Docker Engine installed inside the Linux/WSL2 distribution, and check that `min_rtt` in `samples.csv` matches the path's real RTT.